

Engineering Student Interview

Video Script & Production Notes

Speaker: [Your Name] | Format: Landscape, 5–10 min | Questions covered: 6

Before You Hit Record

This section covers the requirements a script alone can't guarantee — camera setup, framing, and audio. Run through it once before recording, and once again before your final take.

Orientation	Turn your phone or laptop sideways — landscape (horizontal), never portrait. On a laptop webcam this is automatic; on a phone, lock rotation before recording.
Framing	Frame from the chest up, camera at eye level (stack books under a laptop if needed). Center yourself so your full face stays in frame even if you lean or gesture.
Lighting	Face a window or lamp — never sit with a bright light or window behind you, which silhouettes your face. Soft, even light on your face is enough; no ring light required.
Audio	Record in a quiet room with soft surfaces (a bedroom beats a kitchen). Use earbuds with a mic or a headset if you have one — built-in laptop mics pick up more room echo.
Resolution	Record at 1080p or 720p if your device allows it (check camera settings). Keep the lens clean and don't zoom digitally — move the camera closer instead.
Eye Contact	Look at the camera lens, not the screen or your notes. Tape a small arrow near the lens as a reminder, and glance at bullet points only, not full sentences.
Take Length	This script runs about 6–9 minutes depending on your pace. Do one timed practice read before the real take — if you're near either edge (close to 5 or close to 10), adjust your pacing, not the content.

Script

Read this conversationally, not word-for-word — it's written the way you'd naturally speak, so paraphrasing slightly is fine and will sound more authentic than reciting it. Bracketed, italic notes are delivery cues, not spoken text.

Opening (~0:30)

[Look directly at the camera. Sit up straight, relaxed shoulders, easy smile.]

Hi, thanks for having me. My name is [Your Name], and I'm an engineering student here. When your team asked me to sit down and talk through how I've approached college, I actually treated it the way I'd treat any engineering problem: define the constraints, gather data on what's worked and what hasn't, and build a repeatable process out of it. That's basically what I want to walk you through today — five decisions that came out of that process, and why each one mattered.

Q1 — What is the main difference between learning in high school and college? (~1:30)

In high school, the system is built to catch you. Teachers check homework daily, remind you about upcoming tests, and material comes broken into small, graded checkpoints — a tightly controlled feedback loop that someone else manages for you.

College flips that. You might have one homework check and two exams for an entire semester, and nobody is going to remind you a problem set is due. So the real difference isn't that the material is harder — it's that the feedback loop moves from external to internal. You have to build your own checkpoints. I treat every class like a system with delayed feedback: if I wait for the exam to tell me whether I understand thermodynamics, that feedback arrives too late to act on. So I create my own weekly checkpoints — practice sets, quick self-quizzes — so I'm never more than a few days from finding out whether I actually understand something.

Q2 — Why is it important to create a weekly schedule for your activities? (~1:15)

Because time is the one resource in college you can't get more of, and engineers are trained to manage scarce resources deliberately instead of reactively.

At the start of every week, I block out fixed commitments first — classes, labs, work — and then schedule study blocks around them the same way I'd sequence tasks on a project timeline: the hardest, most cognitively demanding work goes in my highest-energy hours, not whatever's left over at 11 p.m. I also build in buffer time, four or five hours a week, because problem sets always take longer than estimated — any engineer who's built a project schedule knows to pad for the unexpected. Without a written schedule, everything competes for "later," and later never comes.

Q3 — Give examples of practices you need to follow to be successful in college. (~1:25)

I'd point to four. First, active recall over re-reading — closing the textbook and forcing myself to reproduce a derivation from memory, because that's the only real test of whether I've learned it, not just recognized it. Second, going to office hours before I'm stuck, not after — treating my professor as a resource to validate my approach early, the same way I'd get a design reviewed before building it. Third, working in study groups, but only after attempting problems alone first, so the session is a real peer review of my work instead of me copying a method I don't understand.

And fourth — engineers tend to underrate this one — protecting sleep and basic health. Fatigue is the single biggest silent killer of problem-solving ability. You can't out-schedule a brain that isn't rested enough to function.

Q4 — Give examples of good note-taking. (~1:15)

I treat notes the same way I'd treat documentation on a project — the goal isn't to capture everything the professor says, it's to leave a reference I can actually use later without re-deriving it from scratch. The method that's worked best for me is splitting the page: worked examples and derivations on one side, and a running column of questions or gaps on the other — the way you'd flag an open issue in a project log instead of just letting it slide. If something doesn't click in the moment, I don't stop and stall the whole lecture in my head; I flag it and keep moving, then close it out later at office hours or in the textbook.

I also rewrite my notes within 24 hours, condensing them into a summary page per topic. That single step does two things at once: it's a form of active recall, since I'm reconstructing the logic instead of just copying it, and it gives me a compressed reference for exam review instead of flipping through a full notebook. Messy notes taken live are fine — the real value gets created in that rewrite step.

Q5 — Why is it important to get involved with an engineering organization? (~1:05)

Because a classroom gives you the theory, but an engineering organization is where you run it through an actual feedback loop. In my club, we build real projects with real constraints — budgets, deadlines, parts that don't arrive on time — and there's no textbook answer key. That's a much closer simulation of actual engineering work than anything in a lecture hall.

It's also where I met upperclassmen and industry mentors who'd already solved the exact problems I was running into — which internship to take, which electives were worth it. Getting involved early compounds: the connections and hands-on reps you build as a freshman are still paying off senior year.

Q6 — Explain why it is important to have a graduation plan. (~1:10)

A degree is really a long project with hard dependencies — prerequisites that gate other prerequisites, courses only offered one semester a year, internship windows that only line up at certain points. If you don't map that out early, you can discover in your junior year that one missing prerequisite just added an entire extra semester.

So I built mine like a project schedule: I mapped every required course backward from graduation, flagged the courses with limited offerings or long prerequisite chains as critical-path items, and left room for an internship and a study term. I revisit it every semester, because plans should update as new information comes in — but having that baseline means I'm making decisions against a plan, not reacting semester to semester.

Closing (~0:30)

[Look at the camera, steady eye contact, slight nod.]

If there's one thread through all of this, it's that I stopped treating college as something that just happens to me and started treating it as a system I could actually design — schedule it, instrument it with feedback, and plan it end to end. That mindset is exactly what's going to carry over into the job. Thanks for having me.

Estimated total runtime: ~6:00–9:00 depending on your pace — comfortably inside the 5–10 minute window even at a fast clip, with room to slow down for clarity and eye contact.

AI Tools for the Parts a Script Can't Fix

Everything above covers content — the 40-point criterion. The items below are technical/production requirements that are faster to solve with software than by re-recording. Pick what you actually need; you don't need all of these.

Auto-captions / subtitles (required by the rubric)

- CapCut — free, generates accurate synced captions automatically and lets you style/edit them; also has an AI auto-reframe tool if you accidentally record portrait.
- Descript — transcribes your video and lets you edit the video by editing the text; very accurate captions, easy to export burned-in or as a file.
- Veed.io — browser-based, one-click auto-subtitles with easy manual correction of any misheard words.
- Kapwing — similar auto-caption workflow, good free tier, works entirely in-browser.
- Adobe Premiere Pro (Text-Based Editing / Auto Caption) — best if you already have access to Premiere; captions sync natively to the timeline.

Audio cleanup (background noise, echo, low volume)

- Adobe Podcast (Enhance Speech) — free web tool, removes room echo/noise and makes a laptop mic sound closer to studio quality in one pass.
- Descript's Studio Sound — one-click noise removal built into the same editor you'd use for captions.

Delivery aid (staying on-camera, not reading off-screen)

- Teleprompter apps (e.g., Parrot Teleprompter, BIGVU) — scroll this script at eye level right above your camera lens so you can read while still appearing to look at the camera.

Framing / format fixes

- CapCut or Veed.io auto-reframe — if you accidentally shoot portrait, these can reframe to landscape, though re-shooting in landscape from the start will always look better.